
TD 9 Protéger une App.

Table des matières

INTRO:.....	Erreur ! Signet non défini.
Environnement de travail	1
Etape 1:	2
Conclusion :	5

INTRO :

Dans le cadre de cette séance de travaux dirigés, nous avons étudié la sécurisation des applications web face aux attaques de type injection SQL. Pour cela, nous avons travaillé dans un environnement virtualisé composé de quatre machines virtuelles interconnectées, simulant un réseau réaliste avec un serveur web, un client légitime, un attaquant et un pare-feu.

Nous avons successivement joué deux rôles : celui de l'hacker, en exploitant une faille SQL sur le site web Mutillidae, puis celui du développeur, en appliquant des contre-mesures de codage sécurisé pour protéger l'application. Ce TP nous a permis de comprendre concrètement les risques liés aux injections SQL et l'importance d'un codage rigoureux pour protéger les données des utilisateurs.

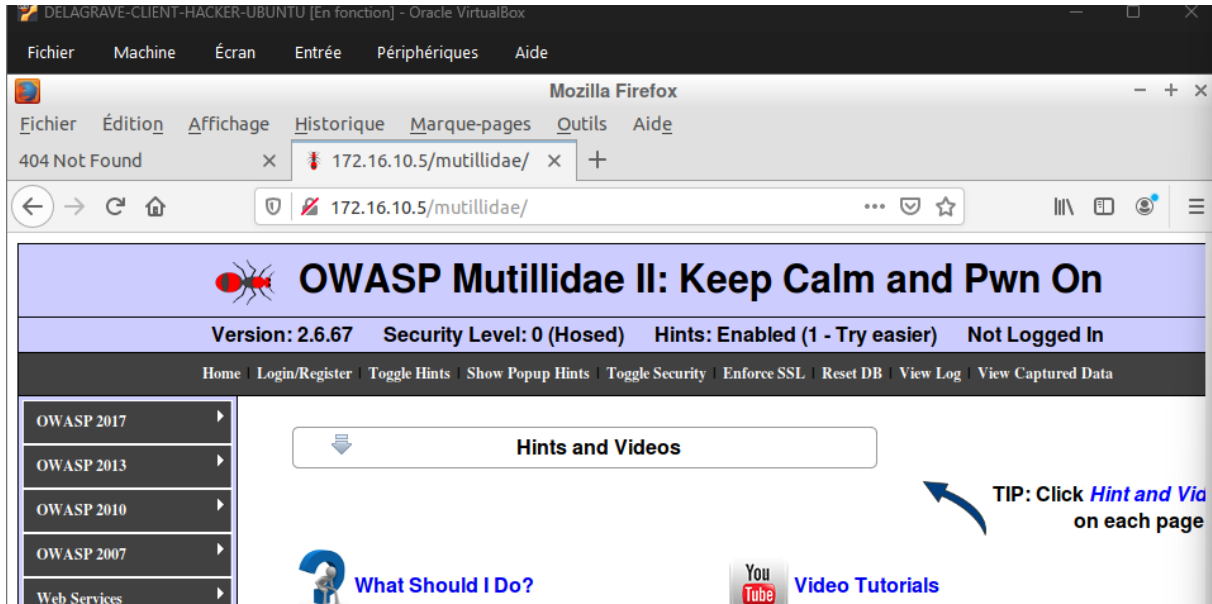
Environnement de travail

Le TP s'effectue sur 4 machines virtuelles sous VirtualBox :

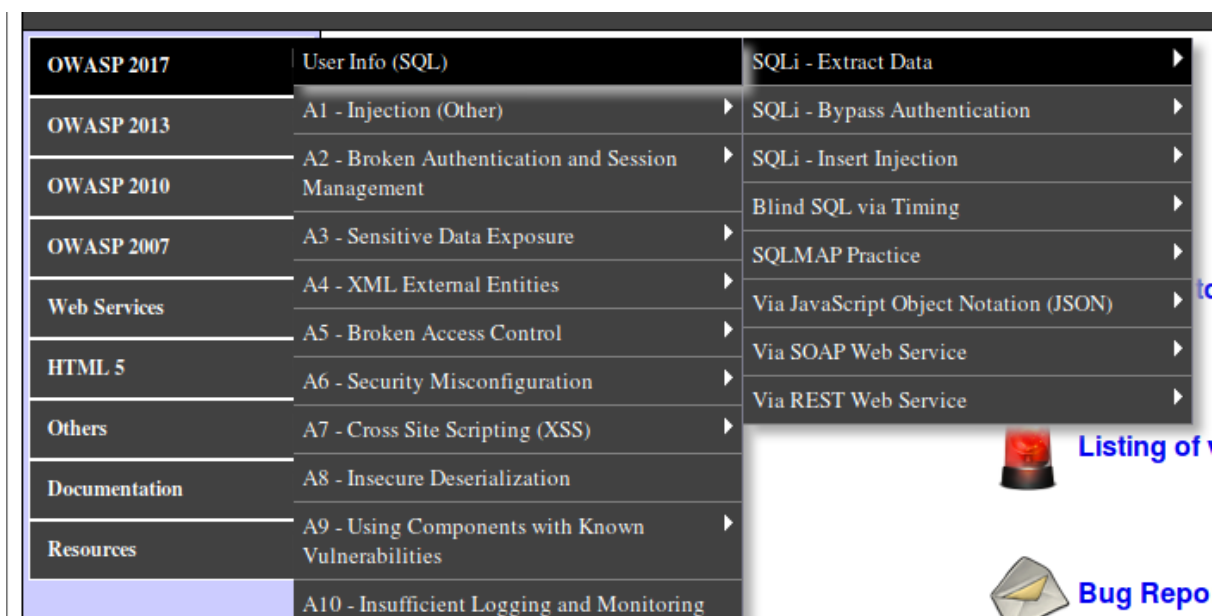
- **Serveur Ubuntu** : héberge le site Mutillidae (172.16.10.5)
- **Client légitime** : simule un utilisateur normal
- **Client hacker** : machine de l'attaquant
- **Firewall pfSense** : assure le routage entre les réseaux

Etape 1:

Nous allons commencer par allumer les machines virtuelles et nous allons aller dans le navigateur de la machine nommé Client-hacker et taper l'adresse suivante <http://172.16.10.5/mutillidae/> pour vérifier le niveau de sécurité.



- Le niveau de sécurité est 0.
- Nous allons nous rendre dans OWASP 2017 pour accéder à la page user-info.php.



- Nous allons faire une injection SQL sur le site avec l'identifiant harry et le mot de passe 'OR'a'='a, nous avons réussi à trouver 23 résultats différents, et cela prouve que le site est vulnérable à une injection SQL.

Dont have an account? [Please register here](#)

Results for "harry".23 records found.

Username=admin

Password=adminpass

Signature=g0t r00t?

Username=adrian

Password=somepassword

Signature=Zombie Films Rock!

Username=john

Password=monkey

Signature=I like the smell of confunk

Username=jeremy

Password=password

Signature=d1373 1337 speak

Username=bryce

Password=password

Signature=I Love SANS

Username=samurai

Password=samurai

Signature=Carving fools

Username=jim

Password=password

Signature=Rome is burning

Username=bobby

Password=password

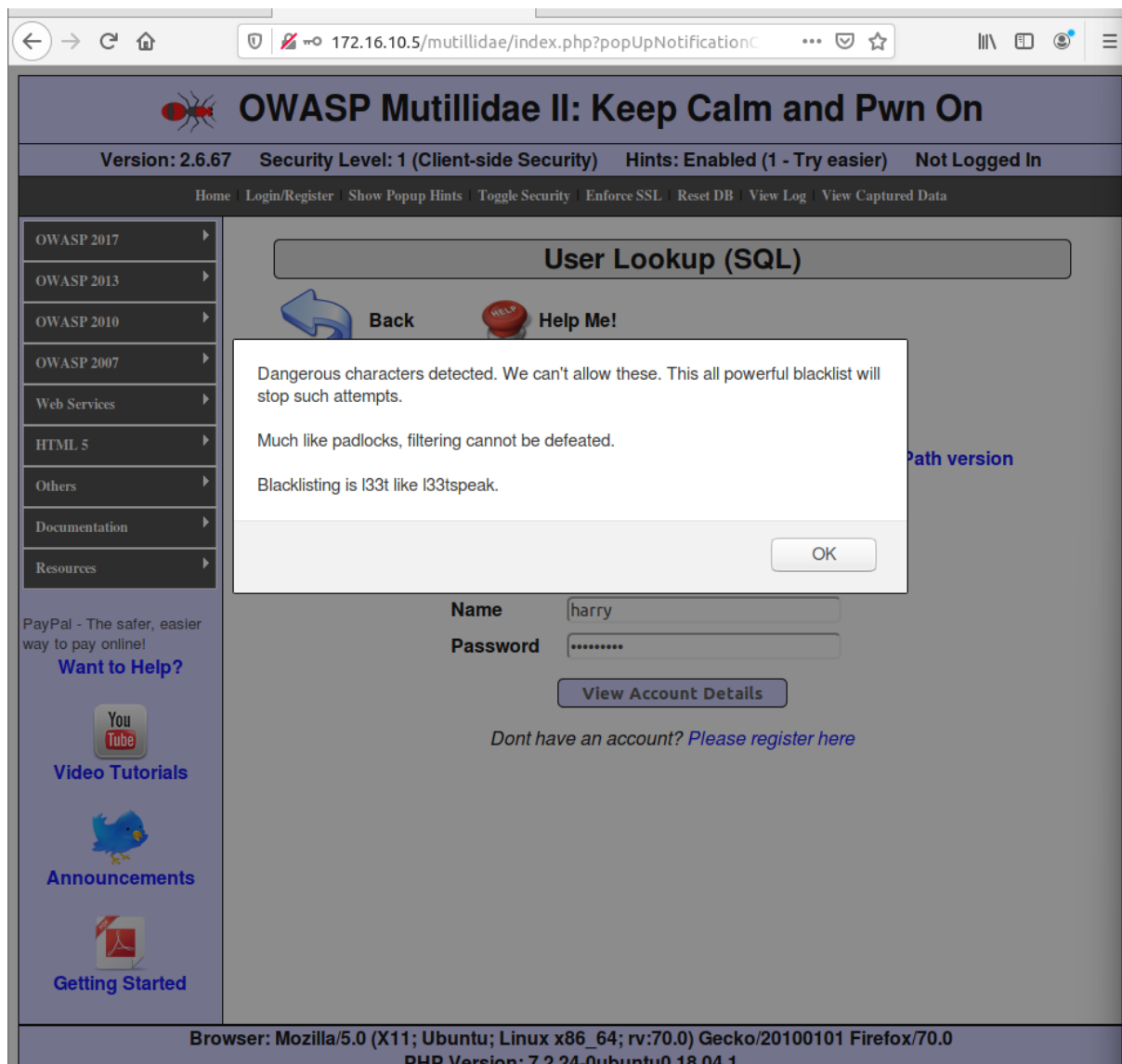
Signature=Hank is my dad

Username=simba

Password=password

Signature=I am a super-cat

- Nous allons changer le niveau de sécurité de la page pour voir les résultats.



- J'ai changé le Security Level de 0 à 1 et cliquant sur toggle sécurité et ça m'a donné comme message quand j'ai essayé d'accéder par les identifiants.
- En passant de niveau de sécurité de 0 à 1 le même injection SQL a été bloquée. Un message d'erreur est affiché (Dangerous characters detected). L'injection SQL ne fonctionne plus, ce qui prouve que le codage sécurisé protège efficacement l'application.
- J'ai pas pu finir ce TP car sur l'ordi il n'y a pas assez de stockage pour installer les machines virtuelles et les autres élèves sont sur leurs ordi où se trouve mon travail.

Conclusion :

Ce TP nous a permis de mettre en évidence de manière concrète la vulnérabilité d'une application web mal sécurisée face aux injections SQL. En effet, au niveau de sécurité 0, l'application Mutillidae était facilement exploitable, permettant l'accès non autorisé à des données sensibles à l'aide de simples requêtes malveillantes.

Cependant, l'augmentation du niveau de sécurité à 1 a montré l'efficacité des mesures de protection mises en place, notamment le filtrage des caractères dangereux, empêchant ainsi l'exécution de l'injection SQL. Cela démontre que la sécurisation du code et la validation des entrées utilisateurs sont indispensables pour protéger une application contre les attaques.

Ainsi, ce travail pratique met en évidence l'importance cruciale du développement sécurisé dans la conception des applications web modernes.